

LAYER 12 OF 13

ERROR TRACKING LOGS

Knowing When Something Breaks (Before Your Users Tell You)

TIER 1 — SOLOPRENEUR

MATT MURPHY .AI

mattmurphy.ai

Part of the Builder Certification Series

What This Kit Covers

What This Kit Covers

This kit covers Layer 12: Error Tracking & Logs—how you find out when your app breaks before your customers do. Every app breaks. That's not a failure—it's just reality. The difference between a professional operation and an amateur one is whether you know about it in thirty seconds or find out three days later when a customer sends an angry email.

You're not going to write error-handling code by hand. You're going to describe your monitoring needs to your AI coding tool and let it wire up error tracking services that watch your app 24/7. Your job is to understand what error tracking does (catches crashes and problems automatically), what logs are (a running diary of everything your app does), and how alerting works (sends you a notification the moment something goes wrong). That's what this certification proves.

This kit walks you through three levels. Tier 1 is for a solo builder who needs to know when their app crashes. Tier 2 is for a growing team that needs organized logging, error grouping, and faster debugging. Tier 3 is for platforms where errors cost real money and you need centralized observability across multiple services. Start at Tier 1—the rest builds on it.

Why It Matters

Right now, if your app crashes at 2 AM, you won't know until someone tells you—maybe a user, maybe a bad review, maybe a client who quietly stops paying. That's not a sustainable way to run a product. Error tracking is your early warning system. It watches your app every second and sends you a message the moment something goes wrong, along with exactly what happened and where.

AI coding tools build features. They don't build monitoring unless you ask. Your AI will happily ship an app with zero error tracking, zero logging, and zero alerting. Then when something breaks—and it will—you'll be debugging blind, guessing where the problem is instead of knowing. This kit teaches you what to ask AI to set up so that when things break, you're the first to know, not the last.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here's a quick look at who each tier is for and what you'll prove you can do.

Tier

Who It's For

What You Prove

Tier 1

Solo builders who need to know when their app breaks

You understand what error tracking is, can have AI set up basic monitoring, and know how to read an error report to understand what went wrong.

Tier 2

Growing teams with apps that need organized logging and faster debugging

You can implement structured logging, group and prioritize errors, set up alerting rules, and use session replay to see exactly what a user experienced.

Tier 3

Platforms with multiple services and strict reliability requirements

You can architect centralized observability across multiple services, implement distributed tracing, define error budgets, and build automated incident response workflows.

TIER 1 CERTIFICATION GOAL

You understand why error tracking matters, can describe what you need to AI, and can read error reports well enough to explain what went wrong—even if you can't fix the code yourself.

What You Need to Know

You don't need to debug code. You need to understand what error tracking tools tell you so you can describe the problem clearly—to AI, to a developer, or in a support ticket.

What an error is: An error is when your app tries to do something and fails. Maybe it tried to load a user's profile from the database and the database was down. Maybe a button triggers code that has a typo. Maybe the app tries to use a feature the browser doesn't support. Errors are normal—every app has them. The question is whether you know about them.

Error tracking services: Tools like Sentry and LogRocket watch your app in real time. When an error happens, they capture it automatically—what went wrong, which page the user was on, what browser they were using, and a stack trace (a breadcrumb trail showing exactly where in the code the error happened). You get a notification instead of silence.

What logs are: Logs are your app's diary. Every time something happens—a user logs in, a payment processes, a page loads—your app can write a log entry. When something breaks, logs tell you the story of what happened right before the crash. Without logs, debugging is guesswork.

Error boundaries (catching crashes gracefully): An error boundary is a safety net built into your app. Instead of the whole app crashing and showing a blank white screen, an error boundary catches the crash and shows a friendly message like "Something went wrong. Please try again." The user gets a bad experience on one part of the page instead of losing the entire app.

The vibecoder monitoring workflow: Tell AI to add error tracking → AI integrates Sentry or a similar service → errors get captured automatically → you receive notifications when something breaks → you read the error report → you describe the problem to AI and ask it to fix it → verify the fix → ship.

Your Toolkit

These tools do the watching for you. Your job is making sure they're connected and you know where to look when they alert you.

Sentry (free tier available): The most popular error tracking tool. It captures errors automatically, groups them by type, and shows you exactly what happened. Tell AI to integrate Sentry into your app—it takes one prompt.

LogRocket or FullStory: Session replay tools that record what a user sees and does. When a user reports a bug, you can watch their session like a video and see exactly what they experienced.

Your platform's built-in logs: Vercel, Railway, Fly.io, and other platforms show you server logs automatically. This is your first place to look when something goes wrong.

Certification Exam Topics

Every exam question is scenario-based. You'll see a real-world situation and need to identify what went wrong or what to set up. Here's what gets tested:

Error awareness: Your app has been live for a month and you've never seen an error report. Does that mean your app has no bugs, or does it mean you have no error tracking?

Error report reading: Sentry shows you an error that says "TypeError: Cannot read properties of undefined." You don't need to fix the code—but can you explain what kind of problem this describes?

User experience during errors: A user sees a completely blank white screen after clicking a button. What's missing that would have shown them a friendly error message instead?

Log usefulness: A user says "the app crashed when I tried to check out." You have no logs. What information are you missing that logs would have given you?

Alert setup: You want to know within five minutes when your app crashes. What do you need to set up to make that happen?

Stack trace basics: An error report shows a stack trace with five lines. What is this telling you and how would you describe it to AI when asking for a fix?

Error grouping: Sentry shows 500 errors but they're actually just three different bugs happening repeatedly. How does error grouping help you focus?

AI integration: You want to add error tracking to your existing app. What would you describe to your AI coding tool to get it set up?

Common Pitfalls

These are the mistakes vibecoders make most often with error tracking and logging. They're easy to make because your app seems fine—until it isn't. Spot them now so you don't find out the hard way.

Not setting up error tracking at all. Your app is crashing and you don't know it. Your users know. They're just not telling you—they're leaving.

Logging sensitive data like passwords, credit card numbers, or personal information. Once it's in your logs, it's a security and privacy violation that can get you in real legal trouble.

Setting up alerts for everything and then ignoring all of them because you're drowning in noise. Alert fatigue is real—if everything is urgent, nothing is urgent.

Not configuring source maps, so every error report shows minified gibberish instead of actual file names and line numbers. You can't debug what you can't read.

Using `console.log` for everything and calling it "logging." `console.log` vanishes when you close the browser. Real logs persist, have severity levels, and are searchable.

Treating all errors as equal priority. A broken image on the About page is not the same as a crashed checkout flow. If you don't set severity levels, you'll waste time on the wrong problems.

Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Do you have an error tracking service (like Sentry) connected to your app right now?

If your app crashed right now, would you get a notification within five minutes—or would you find out from a user complaint?

Can you open your error tracking dashboard and explain what the most recent error means in plain language?

Do your error reports include enough context to understand what the user was doing when the error happened?

Have you told AI to add error boundaries so your users see a friendly message instead of a blank white screen when something crashes?

Are your logs useful—meaning you can search them by user ID, action, or time range—or are they just random text?

Do you know the difference between a critical error (fix it now) and a warning (fix it this week)?

AI Audit Prompt Template

Copy this prompt into your AI coding tool to get a quick health check on your error tracking and logging setup. It checks the same things the certification exam covers.

Review my app's error tracking and logging setup and check the following. For each one, tell me pass or fail with a specific example:

Error tracking integration: Is an error tracking service (like Sentry) connected and capturing errors automatically, or are errors happening silently?

Error boundaries: Are error boundaries set up so users see a friendly message instead of a blank screen when something crashes?

Logging quality: Are logs structured with consistent fields (user ID, action, timestamp, severity) or are they unstructured console.log statements?

Alerting configuration: Are alerts configured so critical errors trigger immediate notifications, or is there no alerting set up?

Source maps: Are source maps configured so error reports show readable file names and line numbers instead of minified code?

Sensitive data protection: Are logs and error reports free of passwords, tokens, credit card numbers, and other sensitive data?

Give me an overall score out of 6 and list the top 3 things to fix first.

What's Next

Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 12 certification exam at your target tier.

The best way to prepare: check your own app right now. Do you have error tracking set up? Intentionally trigger an error and see if your monitoring catches it. Read the error report. Can you understand what happened? Can you describe the problem clearly enough for AI to fix it? That real-world experience is exactly what the exam tests.

CERTIFICATION PATHWAY

Associate Builder: Pass all 13 layers at Tier 1

Certified Builder: Pass all 13 layers at Tier 1 + Tier 2

MADE Certified: Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

MATT MURPHY .AI

mattmurphy.ai

© 2026 Matt Murphy .AI. All rights reserved.